

Lab 10: Encryption Device

Quang Nguyen & Ian Curd

28th April, 2023

Friday lab section

Abstract

The purpose of this lab is to use flip-flops in order to design a state machine. By using a combination of flip-flops and logic gates, a state machine to encrypt a bit stream can be created. The logic gates are designed based on the type of flip-flop and the state graph and the next state table are made from that. Each of the next-state variables then has a k-map for each encrypted digit. This can be used to create the logic equations and the logic gates to make the state machine for encryption.

I. Theory

Identifying the state graph and the implementation of the flip-flops to be used is the main concept for this lab. These are used to remember binary data and have an input used to replace its value or to switch its value according to the condition. The main flip flop used here is the D flip flop whose output copies the input. This makes a very simple truth table and k map since it's just based on the output. The other viable flip flop is the JK flip flop. The input for these are based on if the output changes or not rather than just copying the input like the D flip flop. Depending on how frequently the inputs switch these can often be much more efficient.

II. Methods

The first step in designing the D flip flops for the state machine is to make the next-state table which is from the state graph that satisfies the encryption requirements. Use 6 D flip-flops as having 6 states, from a truth table for all possible input digits on one side and the output digits on the other with $X=0$ or $X=1$ on the bitstream. In this lab it was an encryption device whenever 101 appears to turn into the complemented version or 001 to copy the bitstream again. So need 6 states, which have 3 variables, as it was set up as $ABC \mid A*B*C^*$, there will be 2 values that are don't care terms for each next-state variable. The inputs are all possible numbers as usual and the output side is the completed version or the copy version of the input depending on if 101 or 001 appears. This is clearly a Mealy machine, as present output Z will have to depend on X besides the present inputs.

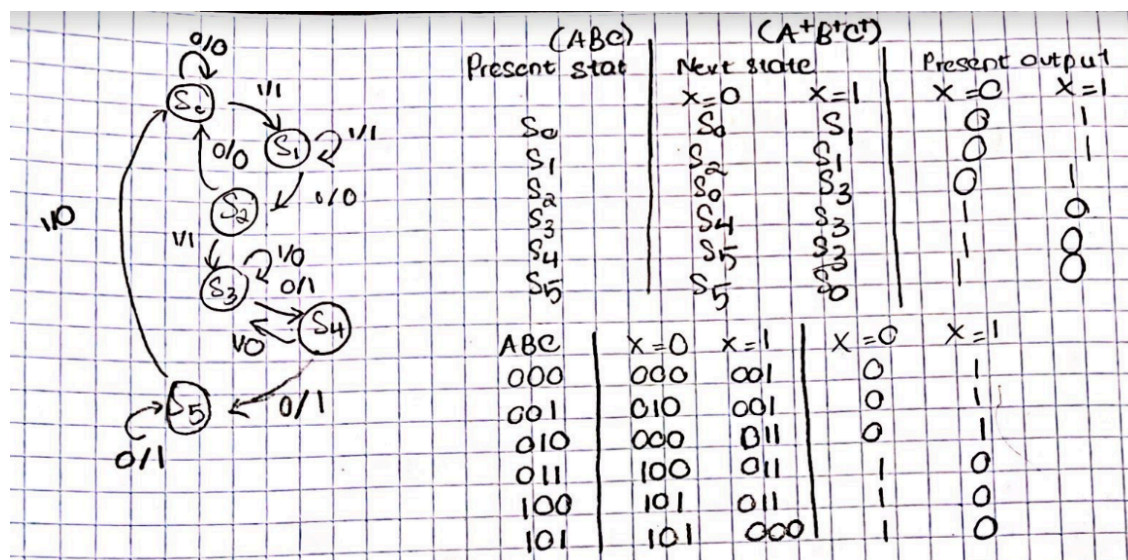
A state graph and the transition table can be constructed, this is then converted into a K map for each of the individual outputs. These are sectioned off to create the logic equations and logic gate configurations as usual. So would have 3 equations for three D flip-flops and one equation for the present output.

The D flip flops only have one influential input which is SW[0], with the clock being the only other as X which is KEY[0]. This input has the logic gate configuration in front of it which is what ensures the encryption goes in the right changes. The inputs are based on the outputs from all of the flip flops so it cycles through easily. The output of each bit goes to the LED in order to have the real world piece of it as it is 1 or 0 after triggering the clock by clicking KEY[0]. The clock input is used to have control over when the sequence is encrypted bit by bit, the button KEY[0] was used here to have manual control.

If JK flip flops had been used then the process would have been similar but instead of the $A*B*C^*$ part of the truth table being would be changed to whether or not the digit switches from the input rather than just the next number in the bitstream. For this lab section sequence only one digit switches at a time so it likely isn't as effective of a solution however it often can be.

III. Data & Analysis

Below is the next-state table, the state graph and the truth table to encrypt the bit stream. As encryption rule changes when 101 or 001 appear, so would need 6 states or 6 D flip-flops to remember these 6 bits 101 or 001, actually divided into 2 groups of 3 D flips, as first will deal with bits before 101 as any bits along the inputting not in this format will just restart the cycle, and the second group deals with the bits before 001. For example 1100 before 101 is received one at a time, 1 will be stored in the second state, 1 is stored again in the second state can't not move to the next state, then 0 is stored in the next state, however the next bit 0 will restart the first group cycle as 101 does not begin with 0. Only when it appears 101 then, then the next group of 3 D flip-flops will be used, which makes every input to be complemented. For example 0001 is input into the state machine bit by bit after 101 is received. 0 is stored as 1 in S4 state, next 0 is stored as 1 in S5 state, next 0 which is not 1 is stored as 1 in S5 again, then 1 will be stored in S0 start the first group as 0 (001) with 001 is achieved, though the output is 1110 after this copy each by bit before receive 001 to start the next group of flip-flops. From these information, a state graph is constructed, then a next-state table can be constructed, then K-maps for each next-state variables and the output can be constructed.



From there, Karnaugh maps are constructed for each of the next state variables and the output. And do the circling of the implicants to reduce the expression to the most simplified version with the least gates and inputs used for each D flip-flop and the output.

For Z

BC \ A	00	01	11	10
00	0	1	0	1
01	0	1	0	1
11	1	0	x	x
10	0	1	x	x

$$Z = A'XB' + B'XC' + AX' + X'BC$$

For A⁺

BC \ A	00	01	11	10
00	0	0	0	1
01	0	0	0	1
11	1	0	x	x
10	0	0	x	x

$$A^+ = AX' + X'BC$$

For B⁺

BC \ A	00	01	11	10
00	0	0	1	0
01	1	0	0	0
11	0	1	x	x
10	0	1	x	x

$$B^+ = AXC' + A'X'B'C + XB$$

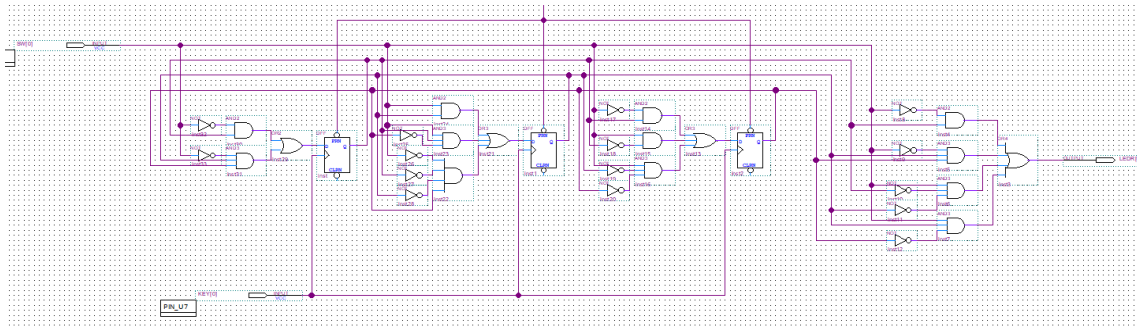
For C⁺

BC \ A	00	01	11	10
00	0	1	1	1
01	0	1	0	1
11	0	1	x	x
10	0	1	x	x

$$C^+ = A'X + \underline{A}X' + XB'C'$$

Schematic diagram

Having three next-state variables corresponds to three D flip-flops and the output to LEDR[0]. Using D flip-flops since they require fewer inputs and gates compared to other types of flip-flops when designing this encryption device. As D flip-flops, the output can only be changed at the clock edge which is the KEY[0]. Which makes spotting the next value from the LED as encrypted output more easily.



For this circuit using D flip-flops and with the K-maps identified above, $A+ = A\sim X + \sim XBC$, $B+ = AX\sim C + \sim A\sim X\sim BC + XB$, $C+ = \sim AX + A\sim X + X\sim B\sim C$, $Z = \sim AB\sim B + BX\sim C + AX\sim + \sim XBC$. Each flip flop will store one bit of information to achieve 101 or 001 or not. This circuits require 3 D-flip flops at the core with other logic gates including AND, OR and inverter gates, as derived from the functions of the next-state K-maps. The circuit above proves to work as intended with the encrypted bit stream. After clicking the clock edge KEY[0], LEDR[0] turns on or off representing the encrypted bit. For example if X bit stream is 1011010011001, the output Z is 1010101101001

Question 4

Given a bitstream in HEX: 4D415920313020383A3330 (MAY{space}10{space}8:30)
The hex number will be equivalent to the below bit stream in binary:
0100 1101 0100 0001 0101 1001 0010 0000 0011 0001 0011 0000 0010 0000 0011 1000 0011
1010 0011 0011 0011 0000

After inputting into the encryption device, this bit stream will encrypt into:
0100 1101 1011 1110 0101 0110 0010 0000 0011 0001 0011 0000 0010 0000 0011 1000 0011
1011 1100 1101 0011 0000

, which is equivalent to in Hexadecimal representation: 4DBE5620313020383BCD30
Then the hexadecimal converted to ASCII code is M¾V 10 8;İ0, the bit stream encrypted seems to have a clear meaning.

IV. Summary

For this lab, guides on using flip-flops and logic gates to develop an encryption device. Since flip-flops can be used to store binary data and these data can be changed by applying varying inputs. Many flip-flops can be grouped to form a common clock input, which is the register. For this lab, a state machine is designed. This can be constructed from multiple flip-flops and logical gates as demonstrated in the lab, that perform the prescribed encryption rule when the input pulse is received each bit by bit. This requires the derivation of the state graph, the next-state table, and from there, construct the flip-flop input equations for the state machine, which can be represented using D-flip flops.